

HDOC Day-16

- **Write a menu-driven C program using switch-case to perform digit based operation on a positive integer. Define one separate user-defined function for each menu option .with void return type and with a simple function call (without call by value/reference).Each function should read the required input, process it, and display the result . Prepare the algorithm and test case table. Then write ,compile, execute and test the C program against the prepared use cases.**
- Check whether its digit are in increasing order from left to right.
- Check whether its digit are in decreasing order from left to right
- Find the largest and smallest digit
- Find the second largest digit and second smallest digit in the number.

Algorithm:

- **Main Algorithm**
 1. Start
 2. Input a positive integer n.
 3. Display the menu
 4. Input the user's choice
 5. Use switch-case:
 - Choice 1 → call increasing(n)
 - Choice 2 → call decreasing(n)
 - Choice 3 → call largestSmallest(n)
 - Choice 4 → call secondLargestSmallest(n)
 - Otherwise display **Invalid choice**
 6. Stop
- **Algorithm for Increasing Order:**

1. Start
2. Extract the last digit using $n \% 10$
3. Remove the last digit using $n / 10$
4. Compare the two adjacent digits
5. If the left digit is greater than or equal to the right digit, the number is not increasing
6. Repeat until all digits are checked
7. Display the result
8. Stop

● **Algorithm for Largest & Smallest:**

1. Set largest = 0 and smallest = 9
2. Extract each digit using $n \% 10$
3. Compare the digit with largest
4. If greater, update largest
5. Compare the digit with smallest
6. If smaller, update smallest
7. Remove the digit using $n / 10$
8. Repeat until $n = 0$
9. Display largest and smallest
10. Stop

● **C Programming:**

- `#include <stdio.h>`
-
- `/* Function to check increasing order */`
- `void increasing(int n)`
- `{`
- `int digit1, digit2;`
- `int flag = 1;`
-
- `while (n >= 10)`
- `{`
- `digit1 = n % 10;`

- n = n / 10;
- digit2 = n % 10;
-
- if (digit2 >= digit1)
- {
- flag = 0;
- break;
- }
- }
-
- if (flag == 1)
- printf("Digits are in increasing order from left to right.\n");
- else
- printf("Digits are NOT in increasing order from left to right.\n");
- }
-
- /* Function to check decreasing order */
- void decreasing(int n)
- {
- int digit1, digit2;
- int flag = 1;
-
- while (n >= 10)
- {
- digit1 = n % 10;
- n = n / 10;
- digit2 = n % 10;
-
- if (digit2 <= digit1)
- {
- flag = 0;
- break;
- }
- }
-
- if (flag == 1)
- printf("Digits are in decreasing order from left to right.\n");
- else
- printf("Digits are NOT in decreasing order from left to right.\n");

- }
-
- /* Function to find largest and smallest digit */
- void largestSmallest(int n)
- {
- int digit;
- int largest = 0;
- int smallest = 9;
-
- while (n > 0)
- {
- digit = n % 10;
-
- if (digit > largest)
- largest = digit;
-
- if (digit < smallest)
- smallest = digit;
-
- n = n / 10;
- }
-
- printf("Largest digit = %d\n", largest);
- printf("Smallest digit = %d\n", smallest);
- }
-
- /* Function to find second largest and second smallest digit */
- void secondLargestSmallest(int n)
- {
- int digit;
- int largest = -1, secondLargest = -1;
- int smallest = 10, secondSmallest = 10;
-
- while (n > 0)
- {
- digit = n % 10;
-
- if (digit > largest)
- {

- secondLargest = largest;
- largest = digit;
- }
- else if (digit > secondLargest && digit != largest)
- {
- secondLargest = digit;
- }
-
- if (digit < smallest)
- {
- secondSmallest = smallest;
- smallest = digit;
- }
- else if (digit < secondSmallest && digit != smallest)
- {
- secondSmallest = digit;
- }
-
- n = n / 10;
- }
-
- if (secondLargest == -1 || secondSmallest == 10)
- {
- printf("Second largest and second smallest digits do not exist.\n");
- }
- else
- {
- printf("Second largest digit = %d\n", secondLargest);
- printf("Second smallest digit = %d\n", secondSmallest);
- }
- }
-
- /* Main function */
- int main()
- {
- int n, choice;
-
- printf("Enter a positive integer: ");
- scanf("%d", &n);

-
- printf("\n----- MENU ----- \n");
- printf("1. Check increasing order\n");
- printf("2. Check decreasing order\n");
- printf("3. Find largest and smallest digit\n");
- printf("4. Find second largest and second smallest digit\n");
- printf("Enter your choice: ");
- scanf("%d", &choice);
-
- switch (choice)
- {
- case 1:
- increasing(n);
- break;
-
- case 2:
- decreasing(n);
- break;
-
- case 3:
- largestSmallest(n);
- break;
-
- case 4:
- secondLargestSmallest(n);
- break;
-
- default:
- printf("Invalid choice!\n");
- }
-
- return 0;
- }

Test Case Table:

Test Case	Input Number	Choice	Expected Output
1	12345	1	Increasing Order
2	54321	1	Not Increasing
3	54321	2	Decreasing Order
4	12345	2	Not Decreasing
5	58321	3	Largest=8,Smallest=1
6	58321	4	Second Largest=5,Second Smallest=2
7	11234	4	Second Largest=3,Second smallest=2
8	987654	4	Second largest= 8, Second smallest= 5

Compilation and Execution:

Test Case 1:

```
(kali㉿kali)-[~]  
$ ./a.out  
Enter a positive integer: 12345  
  
——MENU——  
1.Check increasing order  
2.Check decreasing order  
3.Find largest and smallest digit  
4. Find second largest and second smallest digit  
Enter your choice:1  
Digits are in increasing order from left to right.
```

Test Case 2:

```
(kali㉿kali)-[~]  
$ ./a.out  
Enter a positive integer: 54321  
  
——MENU——  
1.Check increasing order  
2.Check decreasing order  
3.Find largest and smallest digit  
4. Find second largest and second smallest digit  
Enter your choice:1  
Digits are NOT in increasing order from left to right.
```

Test Case 3:

```
(kali㉿kali)-[~]  
$ ./a.out  
Enter a positive integer: 54321  
  
——MENU——  
1.Check increasing order  
2.Check decreasing order  
3.Find largest and smallest digit  
4. Find second largest and second smallest digit  
Enter your choice:2  
Digits are in decreasing order from left to right.
```

Test Case 4:

```
(kali㉿kali)-[~]  
$ ./a.out  
Enter a positive integer: 12345  
  
——MENU——  
1.Check increasing order  
2.Check decreasing order  
3.Find largest and smallest digit  
4. Find second largest and second smallest digit  
Enter your choice:2  
Digits are NOT in drecreasing order from left to right.
```

Test Case 5:


```
Enter a positive integer: 58321

——MENU——
1.Check increasing order
2.Check decreasing order
3.Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice:3
Largest digit = 8
Smallest digit = 1
```

Test Case 6:

```
(kali㉿kali)-[~]
$ ./a.out
Enter a positive integer: 58321

——MENU——
1.Check increasing order
2.Check decreasing order
3.Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice:4
Second largest digit = 5
Second smallest digit = 2
```

Test Case 7:

```
(kali㉿kali)-[~]
$ ./a.out
Enter a positive integer: 11234

——MENU——
1.Check increasing order
2.Check decreasing order
3.Find largest and smallest digit
4. Find second largest and second smallest digit
Enter your choice:4
Second largest digit = 3
Second smallest digit = 2
```

Test Case 8:

```
(kali㉿kali)-[~]  
$ ./a.out  
Enter a positive integer: 987654  
  
——MENU——  
1.Check increasing order  
2.Check decreasing order  
3.Find largest and smallest digit  
4. Find second largest and second smallest digit  
Enter your choice:4  
Second largest digit = 8  
Second smallest digit = 5
```